

MODULE 1

INTERNET OF THINGS

Internet of things: An overview

- Any ‘thing’ that has the ability to connect to the internet and collect and share information and data is a part of **Internet of Things(IoT)**
- IoT is a network of physical objects ("things") embedded with sensors, software, and other technologies to connect and exchange data with other devices and systems over the Internet.
- The ‘things’ that makeup the IoT can be anything from a wearable fitness trackers to an autonomous vehicle . These devices must have the following components:
 - Sensors
 - Actuators
 - IoT gateways
 - The cloud
 - User Interface

- Initially, Radio Frequency Identification (RFID) was the dominant technology behind IoT development.
- Later, Wireless Sensor Networks (WSN) and Bluetooth-enabled devices increased the adoption of IoT.
- IoT creates an intelligent, invisible network fabric. It can be sensed, controlled, and programmed.
- IoT-enabled products use embedded technology.
- This allows them to communicate directly or indirectly. They communicate with each other or through the Internet.

History

- The history of IoT starts with ARPANET, the first connected network.
- In **1982** , a graduate student from Carnegie Mellon University developed a system to monitor the status of **coke vending machine**.
- In **1990** , John Romkey developed **a toaster** that could be turned on and off over the internet – toaster wired to the computer(as no WiFi then!)
- **Kevin Ashton** in 1999, coined the term IoT.
- **1999: Bill Joy** (founder of Sun Microsystems) envisioned device-to-device (D2D) communication as part of his "Six Webs" framework: (Near Web , Here Web , Far Web , Wired Web , B2B , D2D)

IoT Evolution – Three Major Phases

1. Connect the Unconnected

- Connect legacy devices.
- Unite them across the network.

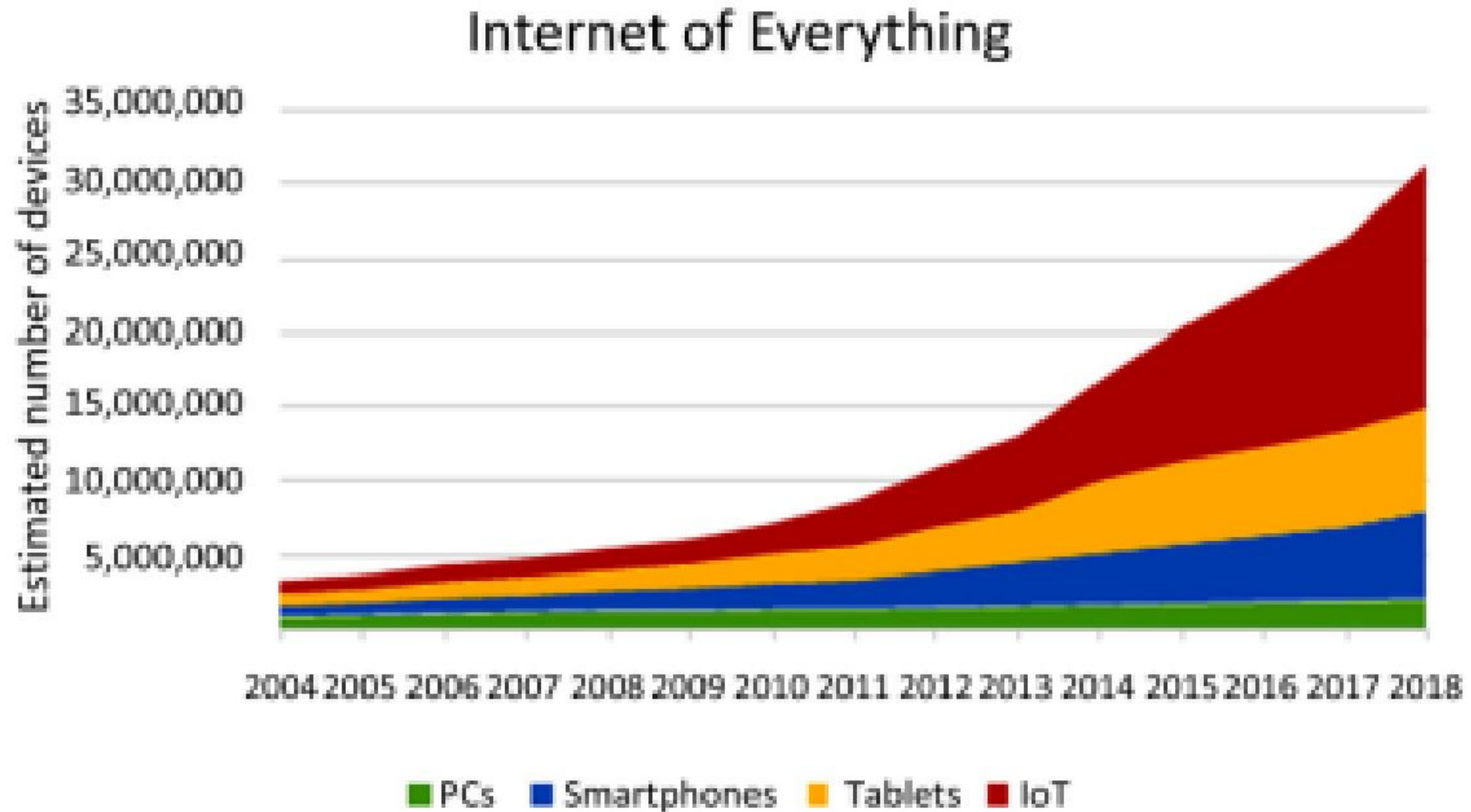
2. Create Smart and Connected Things

- Devices collect data.
- Analyze the data.
- Send important information to the server.

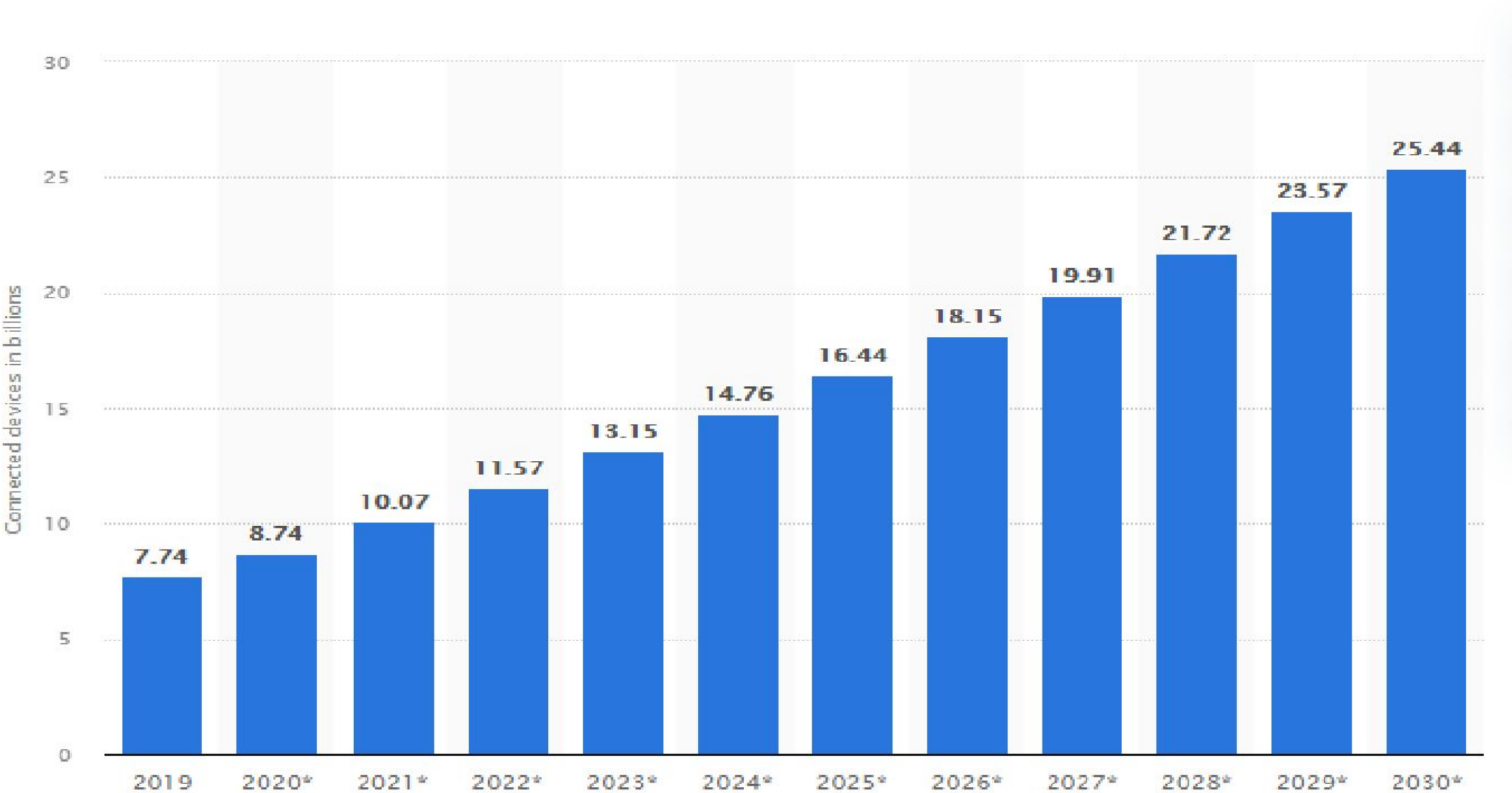
3. Build a Software-Defined Autonomous World

- "Things" become smart enough to make decisions.
- They can operate on their own (autonomously).

IoT Trend and Forecast



Number of Internet of Things (IoT) connected devices worldwide from 2019 to 2030 :



IoT Applications

- IoT has many applications.
- Applications can be developed for almost every industry.
- IoT applications use Artificial Intelligence (AI) and Machine Learning (ML) to make devices intelligent.

Examples of IoT Applications:

- Smart Home
- Health Care
- Traffic Surveillance

1. Smart Home Applications

Smart Home devices collect and share information with one another in an integrated platform and automate their actions based on the owner's preferences.

- **Smart Thermostats:** which monitor and control home temperatures to the comfort of owner.
- **Smart lighting:** lighting adjusts themselves based on the user preferences as well as external lighting.
- **Smart kitchen :** Checks for smoke, carbon monoxide, temperature, and humidity. Monitors whether there are enough products in the refrigerator. Calculates the nutritional value of meals.
- **Security systems:** With special sensors, controllers can automatically lock the door, turn off electronic devices- users can check their home state remotely through app.

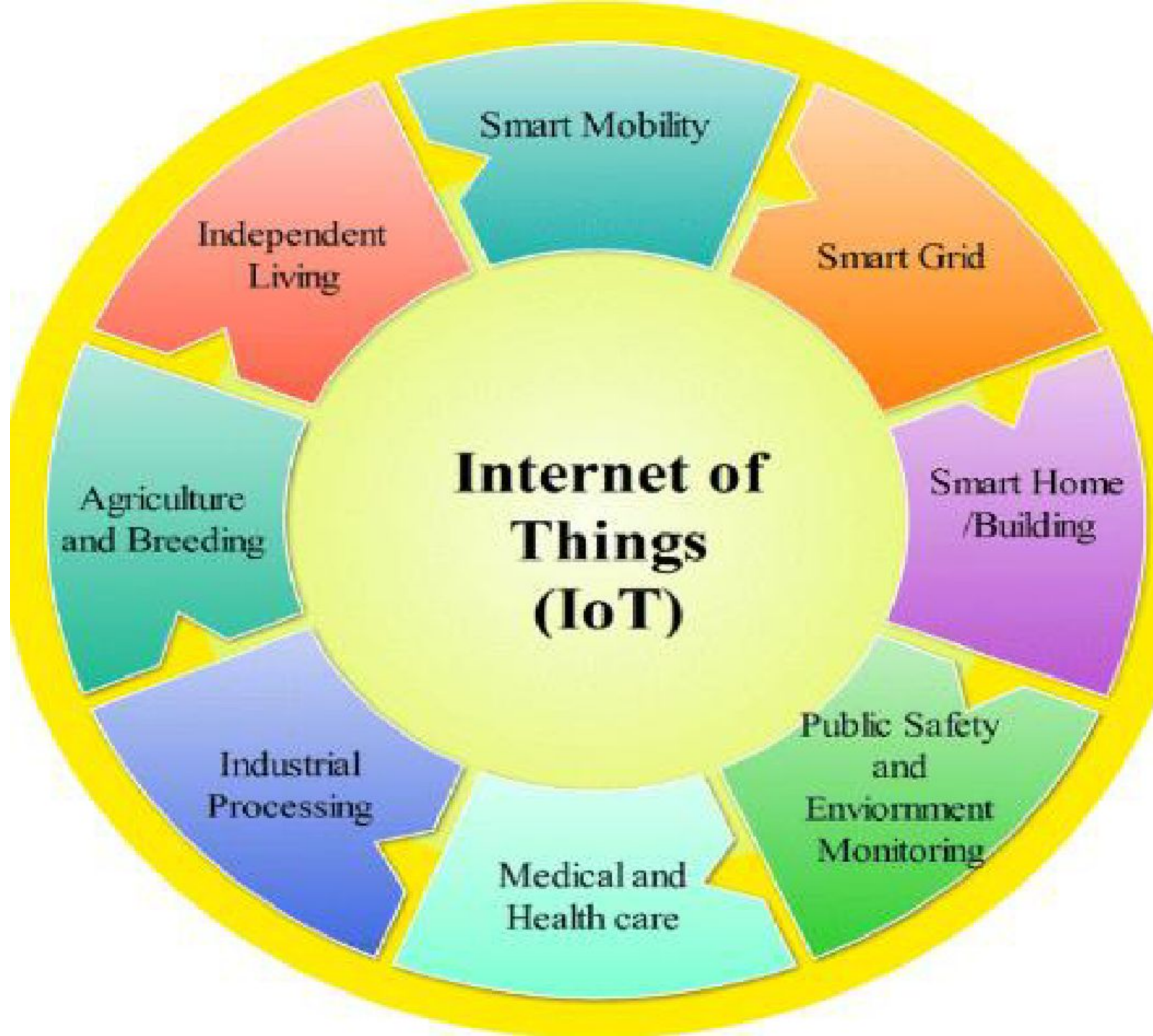
2. Health care applications

- **IoT for patients:** Uses wearable devices such as: Fitness bands, Blood pressure watches, ECG headbands. These devices monitor health conditions.
- **IoT for physicians:** Data collected from the IoT devices can help physicians identify the best treatment process for the patients.
- **IoT for hospitals:** IoT devices with sensors track the real-time location of medical equipment. They also help monitor the deployment of medical staff in real time.
- **IoT for health insurance companies:** Insurance companies use data from health monitoring devices. This helps them detect fraudulent insurance claims.

3. Traffic surveillance

- Smart traffic control
- Vehicle control
- Smart parking
- Connected car
- Drowsiness alerts
- Electronic toll collection system
- Logistics and fleet management

and many others...



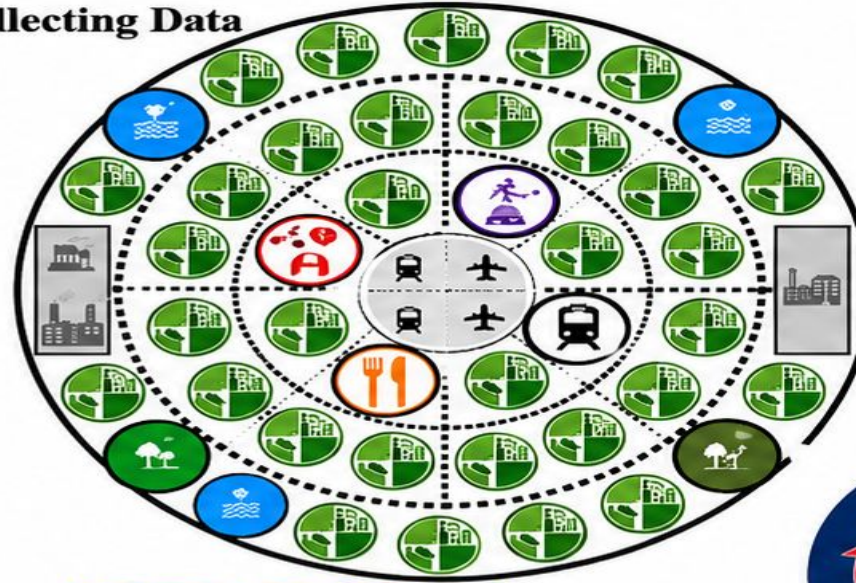
IoT Ecosystem

Sensing and Collecting Data

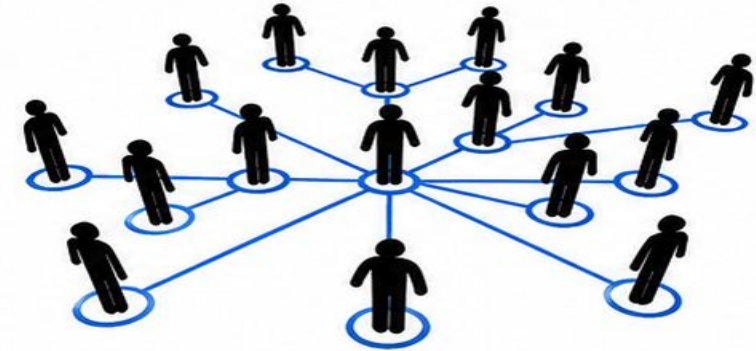
-----> Super speed metro way

- A Single Block
- Water Body
- Green Cover
- Factory
- Railway Station
- Airport
- Recreation Center
- Multi Speciality Hospital
- Multiplex and Restaurant
- Administration and Judiciary

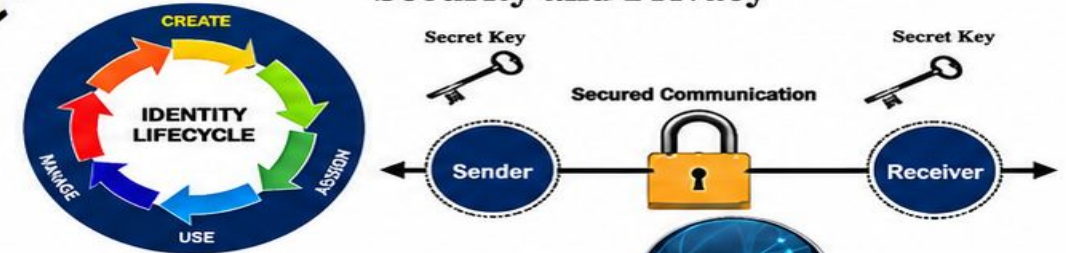
Designed by: Jugal Deshmukh



Keeping Humans in the Loop



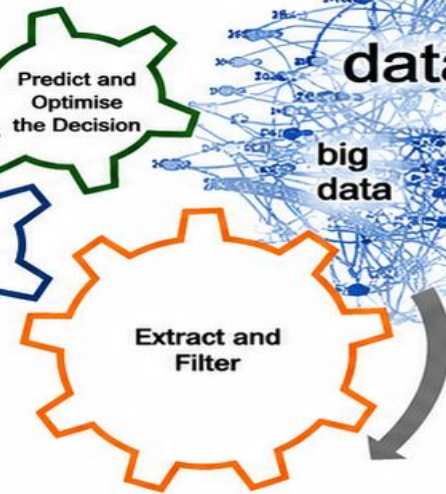
Security and Privacy



data

unstructured data

big data



IoT Ecosystem

Connecting Objects and Humans

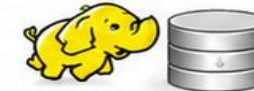
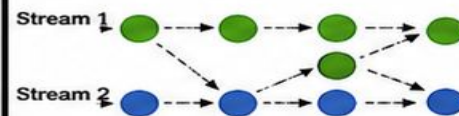
WPAN
(IEEE 802.15.4)

W-Mesh
(IEEE 802.11s)

WLAN
(IEEE 802.11x)

WWAN
(3G/4G/5G)

Data Analytics and Decision Making



Cloud Services



IoT Architectures

- IoT has a wide variety of applications. It works according to the application for which it is designed.
- IoT does not have a standard architecture. The architecture of IoT depends on its functionality and implementation in different sectors.

Building blocks of IoT:

- Sensory devices
- Remote service invocation
- Communication networks
- Context aware processing



A Reference Architecture for IoT

2 Types Of Architecture:

1. SOA Based Architecture
2. API-Oriented Architecture

1. SOA Based Architecture

- Service Oriented Architecture
- SOA ensures interoperability among the heterogeneous devices.
- A generic SOA consists of four layers:
- **Sensing Layer:** It is integrated with available hardware objects to sense the status of things.
- **Network Layer:** It is the infrastructure to support over wireless or wired connections among 'things'
- **Service Layer:** It is to create and manage services required by users or applications.
- **Interface Layer:** It consists of the interaction methods with users or applications.

- In this architecture , a complex system is divided into subsystems that are loosely coupled
- Components can be reused later (modular decomposability).
- If one component fails, the rest of the system continues to operate.
- SOA is widely used in Wireless Sensor Networks (WSN).
- It provides high reliability.
- SOA improves interoperability and scalability.

2. API-Oriented Architecture

- Conventional service-oriented solutions use:
 - SOAP (Simple Object Access Protocol)
 - RMI (Remote Method Invocation)
- These are used to:
 - Describe services
 - Discover services
 - Call services
- SOAP and RMI have high overhead and complexity.
- As an alternative, Web APIs and REST (Representational State Transfer) were introduced.
- During service calls, request–response data conversion happens regularly.
- Lightweight data formats like JSON are used to reduce overhead.

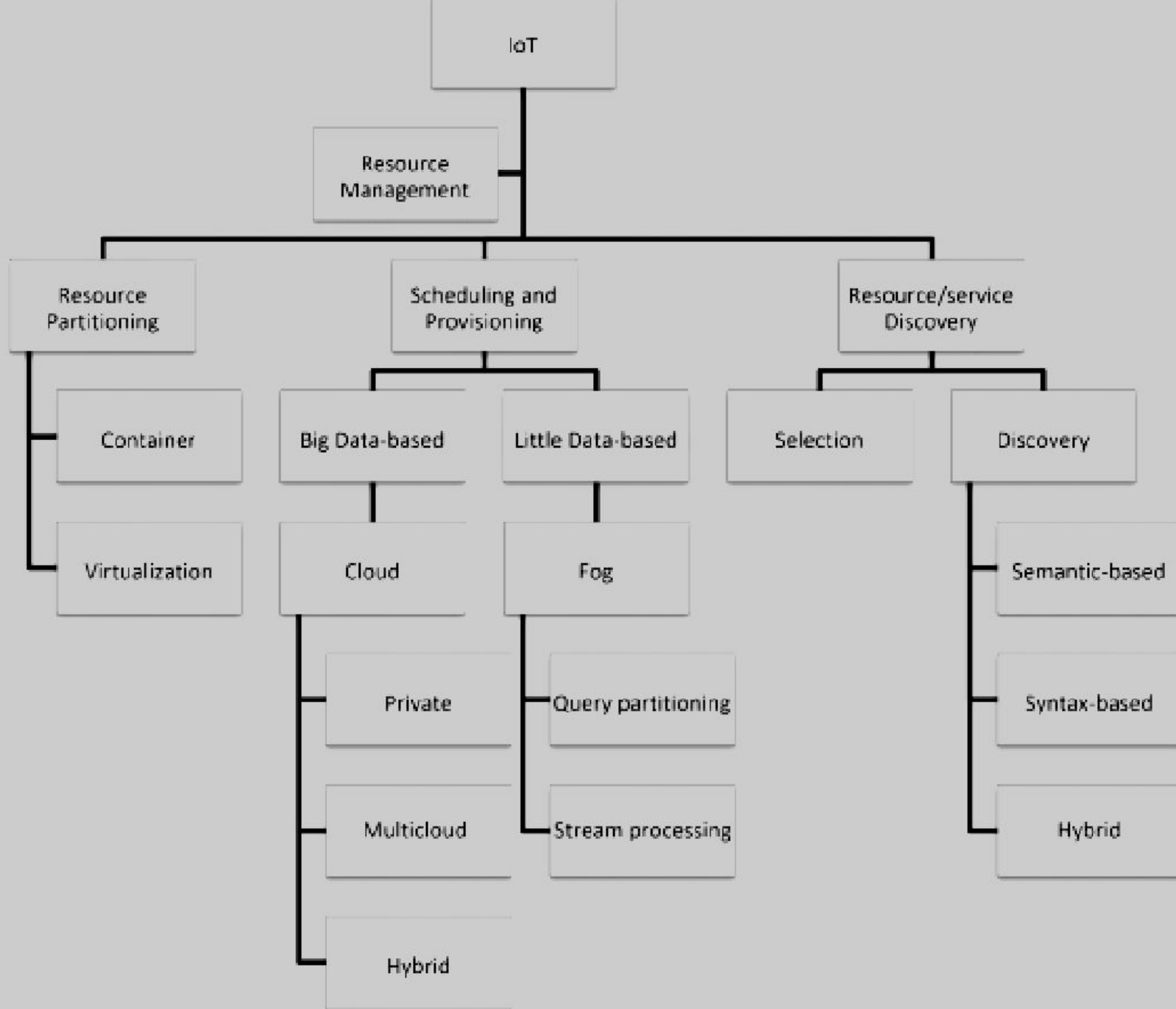
- Modern Web APIs provide security features such as OAuth.
- These security features make it easier to support multitenancy.
- APIs help organizations expose and commercialize their services.
- They provide efficient service monitoring.
- They also offer better pricing tools than previous service-oriented approaches.

Resource Management

- Resource management involves:
 - Discovering and identifying all available resources.
 - Partitioning resources to maximize:
 - Cost efficiency
 - Energy efficiency
 - Performance
- Scheduling tasks on available physical resources.
- Resource management is more challenging in IoT.
- This is because IoT uses heterogeneous (different) and dynamic (changing) resources.

Efficient resource management module needs:

- Robustness
- fault-tolerance
- Scalability
- energy efficiency
- QoS(Quality of Service)
- SLA(Service Level Agreement)



Taxonomy of Resource Management in IoT

Resource Partitioning

- Resource partitioning helps use resources efficiently.
- It improves the resource utilization rate.
- In cloud computing, this is done using:
 - Virtualization techniques
 - Commodity infrastructures
- Virtualization requires:
 - High memory
 - High computational power
- This is not suitable for IoT devices because they have:
 - Limited memory
 - Limited processing power
- To overcome this problem, Containers are used.
- Containers are a new form of virtualization technology suitable for resource-constrained IoT devices.

Computational Offloading (Code Offloading)

- Code offloading (Computational Offloading) is a solution for the limited resources of mobile and smart devices.
- It improves:
 - Power management
 - Storage efficiency (less storage required)
 - Application performance
- Developers manually identify (annotate) the functions to be executed on another device.
- The combination of Virtual Machines (VMs) and mobile clouds provides:
 - Code sharing
 - Code synchronization
 - Code execution on different platforms.

Identification and Resource Discovery

The discovery module in IoT consists of two steps:

1. Identify and Locate the Device

- Identify and locate the actual device.
- This is done by storing and indexing metadata of each object.

2. Discover the Target Service

- Discover the service that needs to be invoked (used).
- Without an effective discovery algorithm, IoT may face:
 - Execution delays
 - Poor user experience
 - Runtime failures
- Efficient discovery algorithms can:
 - Dynamically choose centralized or flooding strategies.
 - Reduce energy consumption.

IoT Data Management and Analytics

- IoT generates huge amounts of data.
- This data comes from a distributed and heterogeneous environment.
- Special methods are needed to process and manage this data.
- Big Data procedures must be updated for IoT, including:
 - Data acquisition
 - Data filtering
 - Data transmission
 - Data analysis

3Vs of Big Data:

- Velocity – Speed of data generation and processing.
- Volume – Large amount of data.
- Variety – Different types and formats of data.

IoT Data Processing Approaches

- Focusing on one or more of the 3Vs of Big Data has led to different data-processing approaches.
- Methods Used for Data Analysis
 - Batch Processing
 - Stream Processing
- Applications use pattern detection and data-mining techniques to:
 - Extract knowledge.
 - Make smarter decisions.
- The centralized nature of these algorithms:
 - Affects performance.
 - Makes them unsuitable for geographically distributed and heterogeneous IoT environments.

IoT and the Cloud

- Cloud computing is used to analyze data generated by IoT objects.
- Data can be analyzed in:
 - Batch format
 - Stream format
- The pay-as-you-go model of cloud providers:
 - Reduces the cost of computing
 - Reduces the cost of data storage
 - Reduces the cost of data analysis
 - Simplifies the development of IoT applications
- Cloud elasticity enables distributed Stream Processing Engines (SPEs) to provide:
 - Fault tolerance
 - Auto scaling for bursty workloads

- Most cloud–IoT integration approaches use a three-tier architecture.
- Three Layers
 - Bottom Layer: Consists of IoT devices.
 - Middle Layer: Consists of the cloud provider.
 - Top Layer: Hosts applications and high-level protocols.
- This approach limits the practicality and full utilization of cloud computing in scenarios where minimizing end-to-end delay is important.
- Example: Online game streaming.

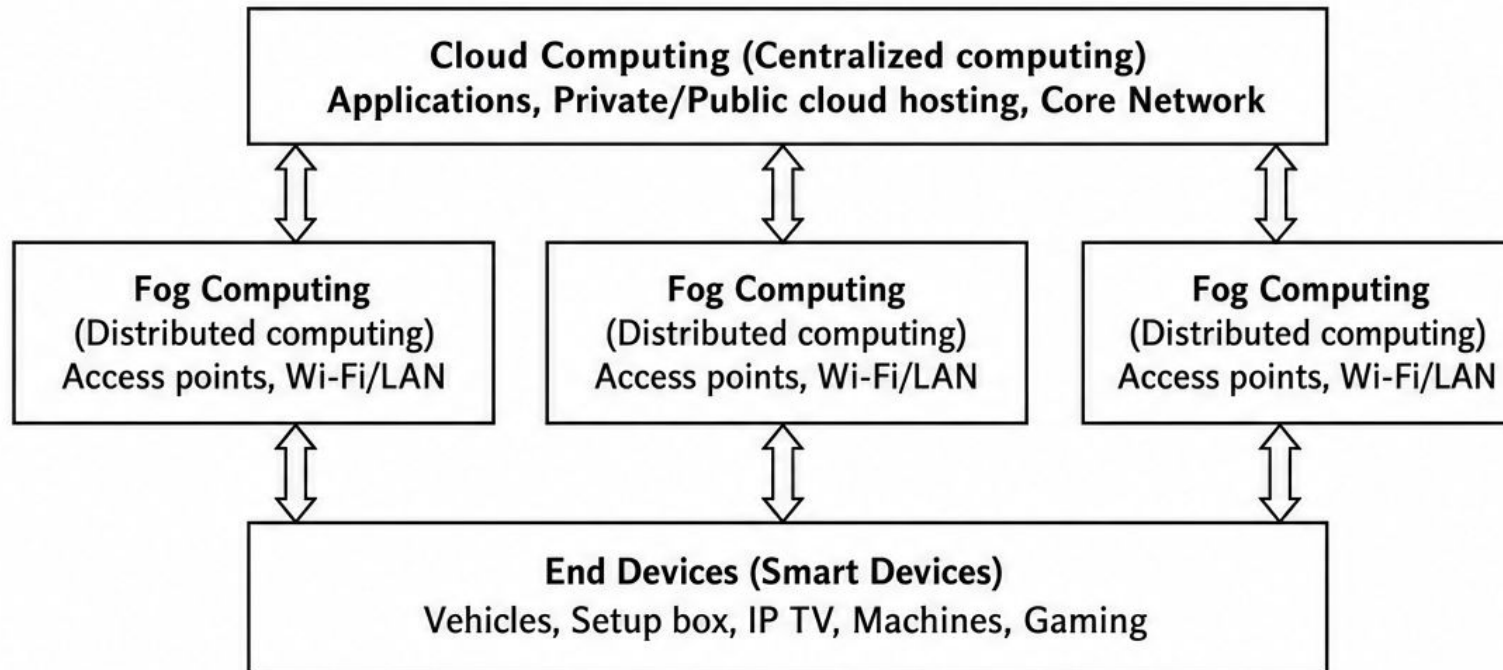
Real-Time Analytics in IoT and Fog Computing

- Current data analytics mainly focuses on Big Data.
- Processing data from millions of sensors and devices in real time is challenging.
- The need for real-time processing and increased computational power of edge devices has led to Edge Computing.
- Edge devices include:
 - Routers
 - Switches
 - Access points
- The Edge layer contains devices that are closer to the end user than the application servers.
- Examples of edge devices:
 - Smartphones
 - Smart TVs
 - Network routers

Fog Computing

- Fog Computing is an extension of Cloud Computing.
- It provides the same features as Cloud Computing, such as:
 - Networking
 - Computation
 - Virtualization
 - Storage
- It also supports applications that require:
 - Low latency
 - Specific Quality of Service (QoS)
 - Service Level Agreement (SLA) requirements
- Fog Computing helps in developing:
 - Mobile applications
 - Geo-distributed applications (e.g., WSN)
 - Large-scale monitoring and control systems (e.g., surveillance camera networks)

Cloud Versus Fog		
Feature	Fog	Cloud
Response time	Low	High
Availability	Low	High
Security level	Medium to hard	Easy to medium
Service focus	Edge devices	Network/enterprise core services
Cost for each device	Low	High
Dominant architecture	Distributed	Central/distributed
Main content generator—consumer	Smart devices—humans and devices	Humans—end devices



Typical Fog Computing Architecture

Open IoT Architecture for IoT/Cloud Convergence

Sensor Middleware

- Collects, filters, and combines data streams from:
 - Virtual sensors (e.g., signal-processing algorithms, information-fusion algorithms, social media data streams)
 - Physical sensing devices (e.g., temperature sensors, humidity sensors, weather stations)
- Acts as a hub between the Open IoT platform and the physical world.
- Enables access to real-world information.
- Provides an interface to:
 - Physical sensors
 - Virtual sensors
 - Data streams from other IoT platforms
 - Social networks

Cloud Computing Infrastructure

- Stores data streams received from the Sensor Middleware.
- Acts as a cloud database.
- Stores metadata for various services as part of the scheduling process.
- Can be:
 - Public cloud (e.g., Amazon EC2)
 - Private cloud (e.g., OpenStack-based private cloud)
- Functions as a Sensor Cloud because it mainly supports:
 - Storage of sensor data streams
 - Management of sensor data streams and their metadata

Directory Service

- Stores information about all sensors available in the Open IoT platform.
- Provides facilities to:
 - Register sensors
 - Discover sensors
- Uses semantically annotated descriptions of sensors.
- Is deployed within the Cloud Infrastructure.
- Provides access to:
 - Sensor data
 - Sensor metadata stored in the cloud.

Global Scheduler

- Processes requests for on-demand deployment of services.
- Ensures services have access to the required resources (e.g., data streams).
- Parses the service request.
- Discovers the sensors needed to fulfill the request.
- Selects the required resources (sensors) for service deployment.
- Performs the reservation of resources.
- Enables the scheduling of all IoT services.

Local Scheduler

- Executes at the Sensor Middleware level.
- Ensures optimized access to the resources managed by the Sensor Middleware.
- Regulates the access and use of data streams at the Sensor Middleware level.
- Works with the Global Scheduler:
 - Global Scheduler manages resources of the Open IoT platform.
 - Local Scheduler manages data streams at the Sensor Middleware level.

Service Delivery and Utility Manager

Performs two roles:

1. Service Delivery

- Combines data streams based on service workflows.
- Delivers the requested service.
- Uses:
 - Service description
 - Resources identified and reserved by the Scheduler

2. Utility Management

- Tracks utility metrics for each service.
- Supports:
 - Accounting
 - Billing
 - Utility-driven resource optimization
- Essential for the pay-as-you-go computing model.

Request Definition Tool

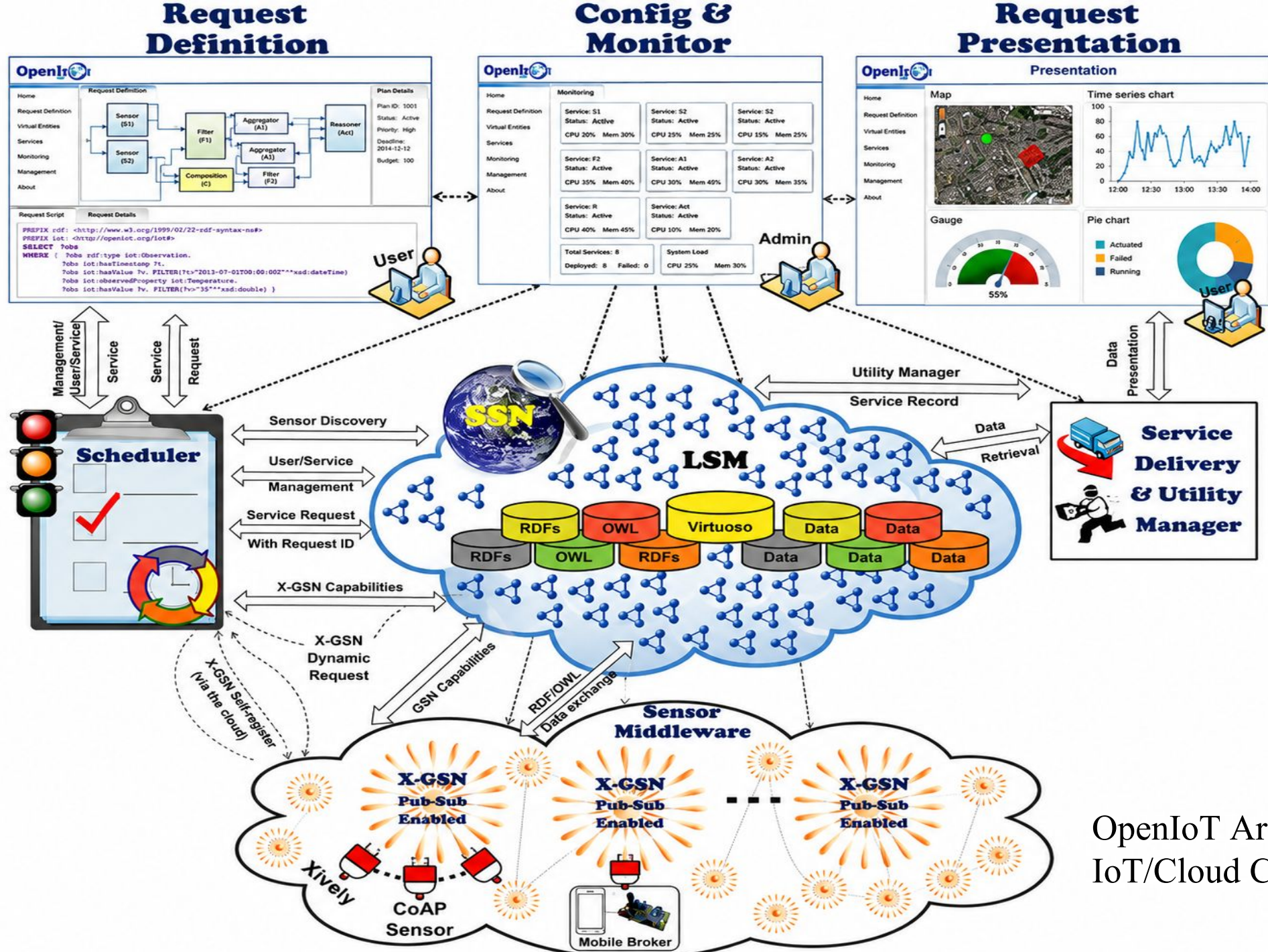
- Enables users to specify service requests to the Open IoT platform.
- Provides services to:
 - Specify service requests.
 - Formulate service requests.
 - Submit requests to the Global Scheduler.
- Includes a Graphical User Interface (GUI).

Request Presentation Component

- Responsible for the visualization of IoT service outputs.
- Selects mashups from an appropriate library.
- Helps in service presentation.

The Configuration and Monitoring component

- It enables management and configuration functionalities over the sensors.
- It enables IoT services that are deployed within the platform.
- This component is also supported by a GUI.



OpenIoT Architecture for IoT/Cloud Convergence

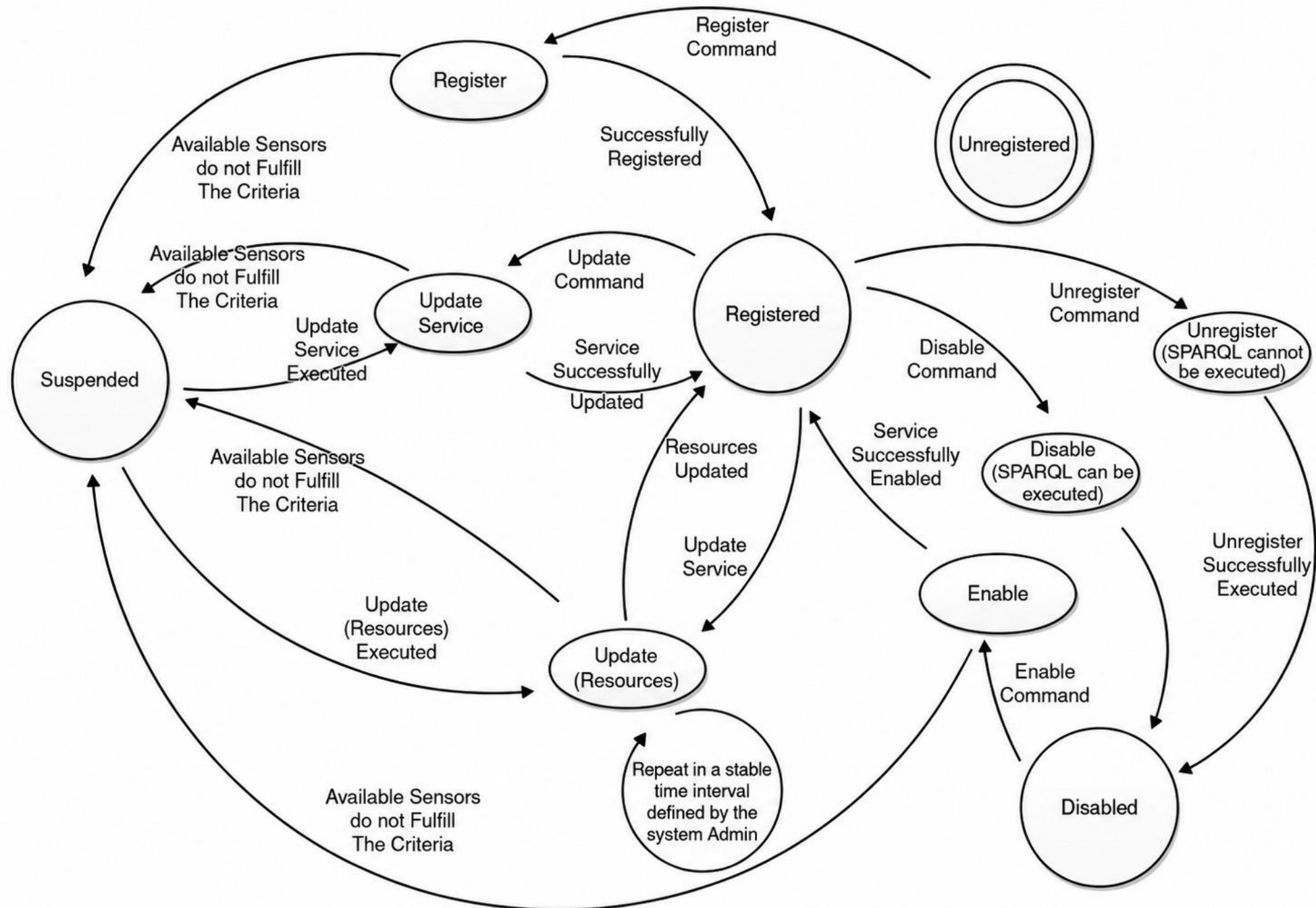
Workflow of an Open IoT Platform

- A service request is created using the Request Definition Tool.
- The request is submitted to the Global Scheduler.
- The Global Scheduler:
 - Parses the service request.
 - Discovers the required sensors/ICOs.
 - Accesses the Directory Service to find the required sensors.
- The service is formulated (e.g., as a SPARQL query).
- The service and its metadata are stored in the cloud.
- End users execute the service using the service handle.
- The results are visualized for the end users.

Note: **ICO** stands for Internet Connected Object, which is any IoT device or sensor connected to the Internet.

Scheduling Process and IoT Services Lifecycle

- The Global Scheduler is the main entry point for service requests submitted to the cloud platform.
- It parses each service request.
- It performs two main functions:
 - Selects the required sensors.
 - Reserves the required resources.
- The Global Scheduler manages the metadata of IoT services, including:
 - Service signature (input and output parameters).
 - Sensors used to deliver the service.
 - Execution parameters, such as:
 - Service repetition intervals.
 - Types of visualization.
 - Other resources used by the service.



State Diagram of the OpenIoT Services Lifecycle Within the Scheduler Module

Lifecycle Management Services

1. Resource Discovery

- Discovers the availability of virtual sensors.
- Provides the resources required for an IoT service request.

2. Register

- Establishes the requested service in the cloud database.
- Assigns a unique identifier (ServiceID) to the IoT service.
- Stores the metadata of each registered IoT service.

3. Unregister

- Releases the resources allocated to the IoT service.
- Identifies the service using its ServiceID.
- Updates the service status in the metadata.

4. Suspend

- Deactivates the IoT service.
- Stops the service operation.
- Does not release the resources associated with the service.

5. Enable from Suspension

- Reactivates a previously suspended service.
- Updates the service metadata stored in the cloud.

6. Enable

- Enables an unregistered service.
- Registers the service again in the platform.
- Identifies and stores the required sensors.

7. Update

- Allows changes to an IoT service.
- Updates the service's lifecycle metadata, including:
 - Service signature
 - Sensors
 - Execution parameters
- Updates the service metadata with the new information.

8. Registered Service Status

- Provides the lifecycle status of an IoT service.
- Identifies the service using its ServiceID.
- Displays detailed information (metadata) about the service.

9. Service Update Resources

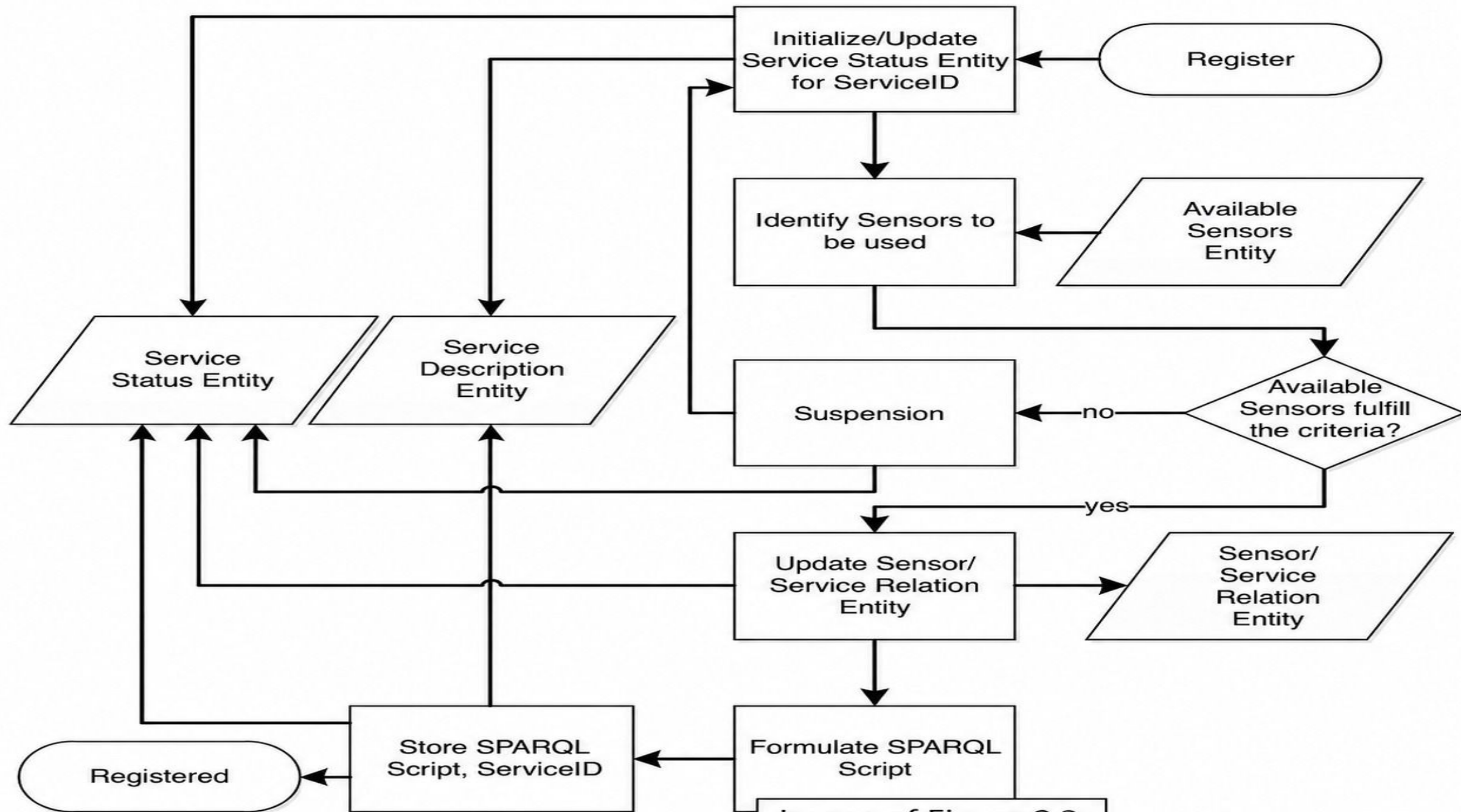
- Periodically checks all enabled services.
- Identifies services using mobile sensors (e.g., smartphones, UAVs).
- Updates the IoT service metadata based on the newly identified sensors.

10. Get Service

- Retrieves the description of a registered service.
- In OpenIoT, it retrieves the SPARQL description of the service.

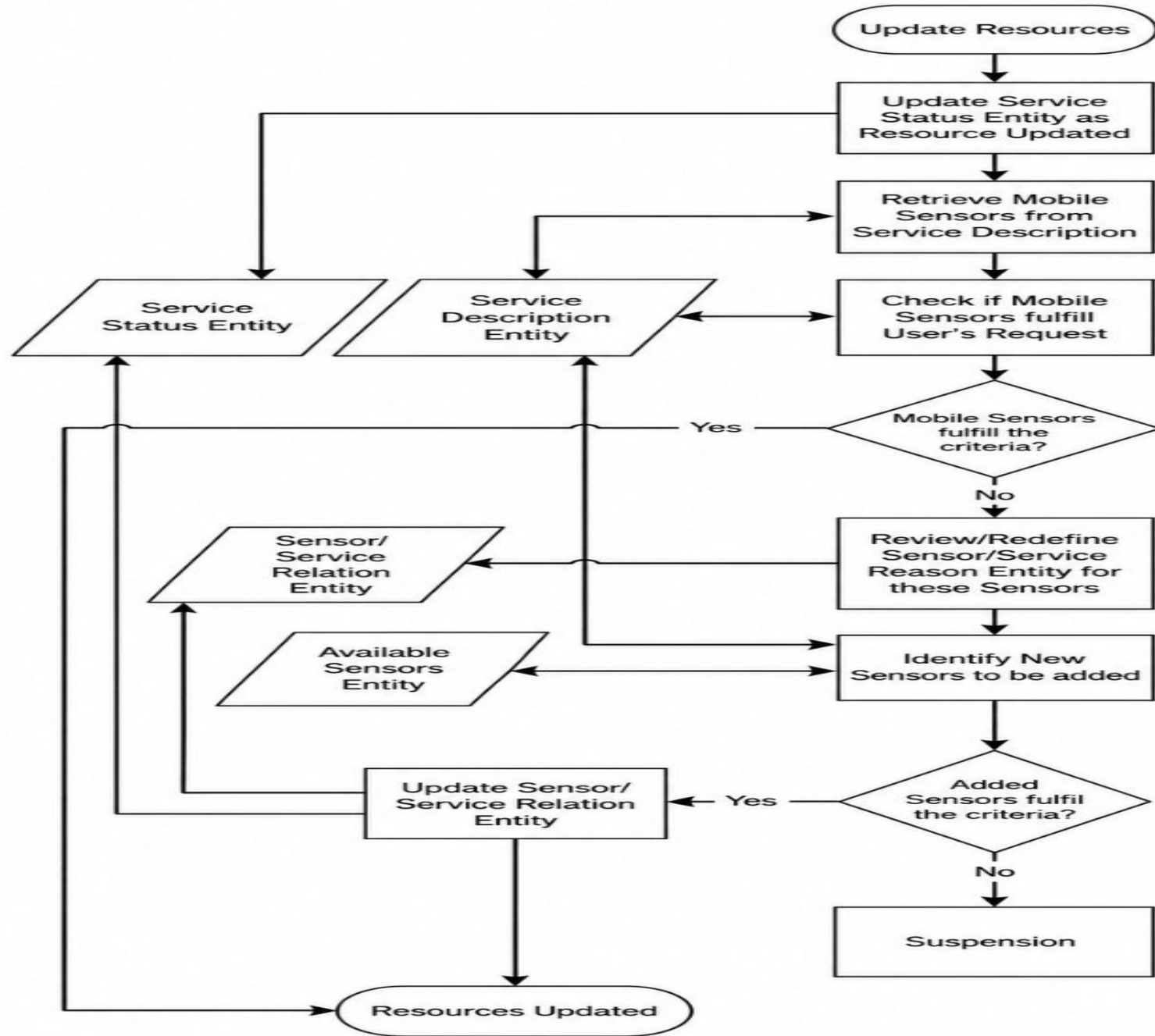
11. Get Available Services

- Returns a list of registered services associated with a particular user.
- These services are registered and established by platform users.

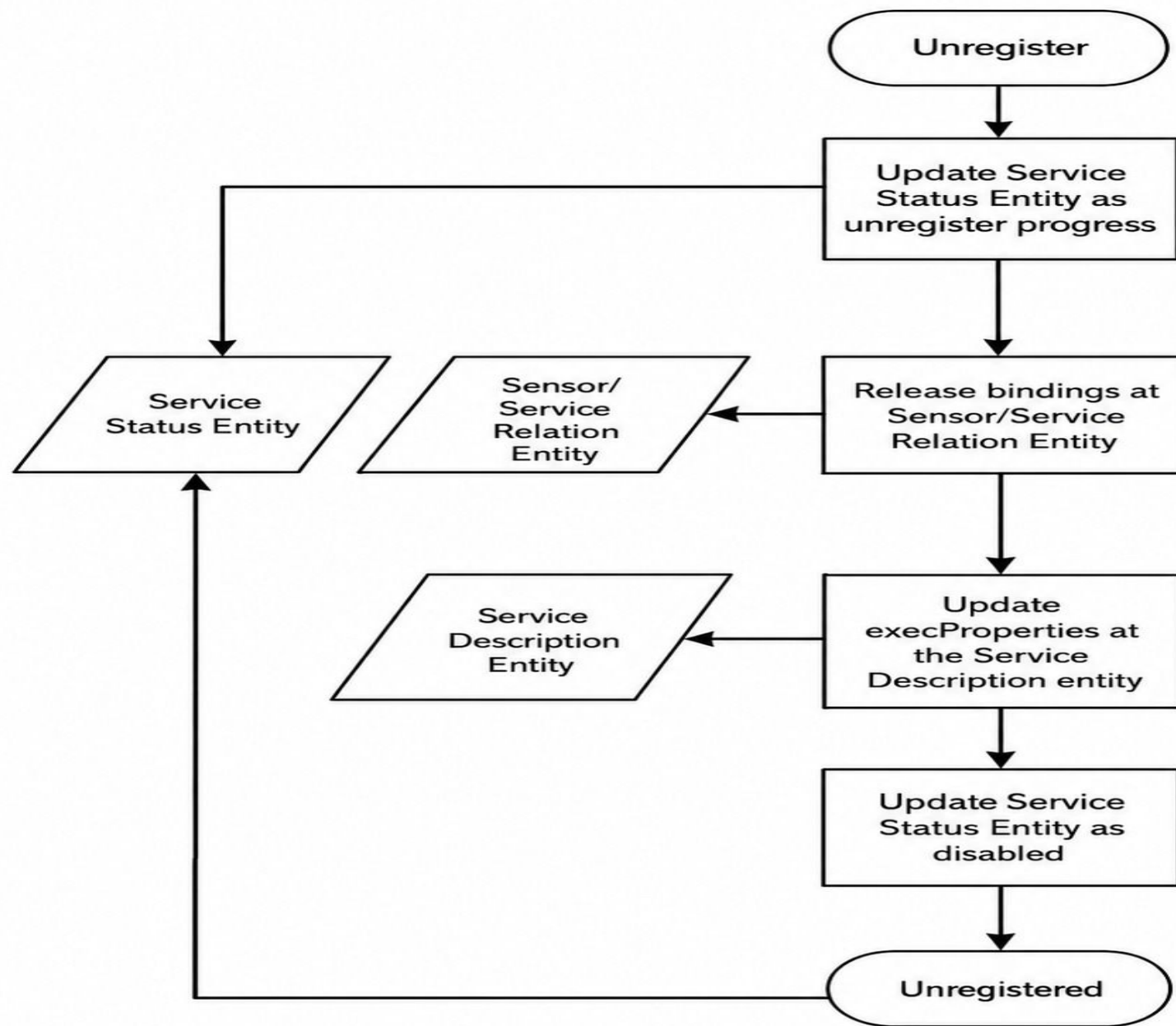


“Register Service” Process Flowchart

Image of Figure 2.3



“Update Resources” Service Flowchart



"Unregister Service" Service Flowchart

Scheduling and Resource Management

- The OpenIoT Scheduler provides accurate information about:
 - The data requested by each service.
 - The sensors used to deliver the data.
- Different resource management and optimization algorithms can be implemented in the Scheduler.
- Local-level scheduling (at the Sensor Middleware) enables resource optimization during sensor data acquisition.

Resource Optimization Scheme

- Uses in-network processing and data management techniques:
 - Push approach
 - Pull approach
 - Hybrid approach
- These techniques help to:
 - Optimize processing time
 - Reduce access to the sensor network
- Query and result aggregation can be based on common spatial regions.
- Data can be aggregated from sensors located in geographical regions of high interest.
- Caching techniques are another optimization method supported by the scheduling approach.

- The first optimization scheme focuses on:
 - Bandwidth optimization
 - Storage optimization
- It uses indirect control over the sensors.
- It follows the in-network processing approach.
- A pull approach is used to access the sensor networks.
- A periodic task runs on the X-GSN module for sensor management.
- The task queries the LSM/W3C SSN repository to identify the sensors required by IoT services.
- The task compares the query results (LSM triplets) with the list of currently active sensors in the X-GSN Sensor Middleware module.

X-GSN:

- Activates sensors that are needed by IoT services but are not currently active.
- Deactivates sensors that are active but not used by any IoT service.
- This ensures that only necessary sensor data is sent to the cloud.
- It helps to:
 - Reduce bandwidth usage
 - Reduce cloud access costs
- This pull approach helps optimize:
 - Latency
 - Monetary costs

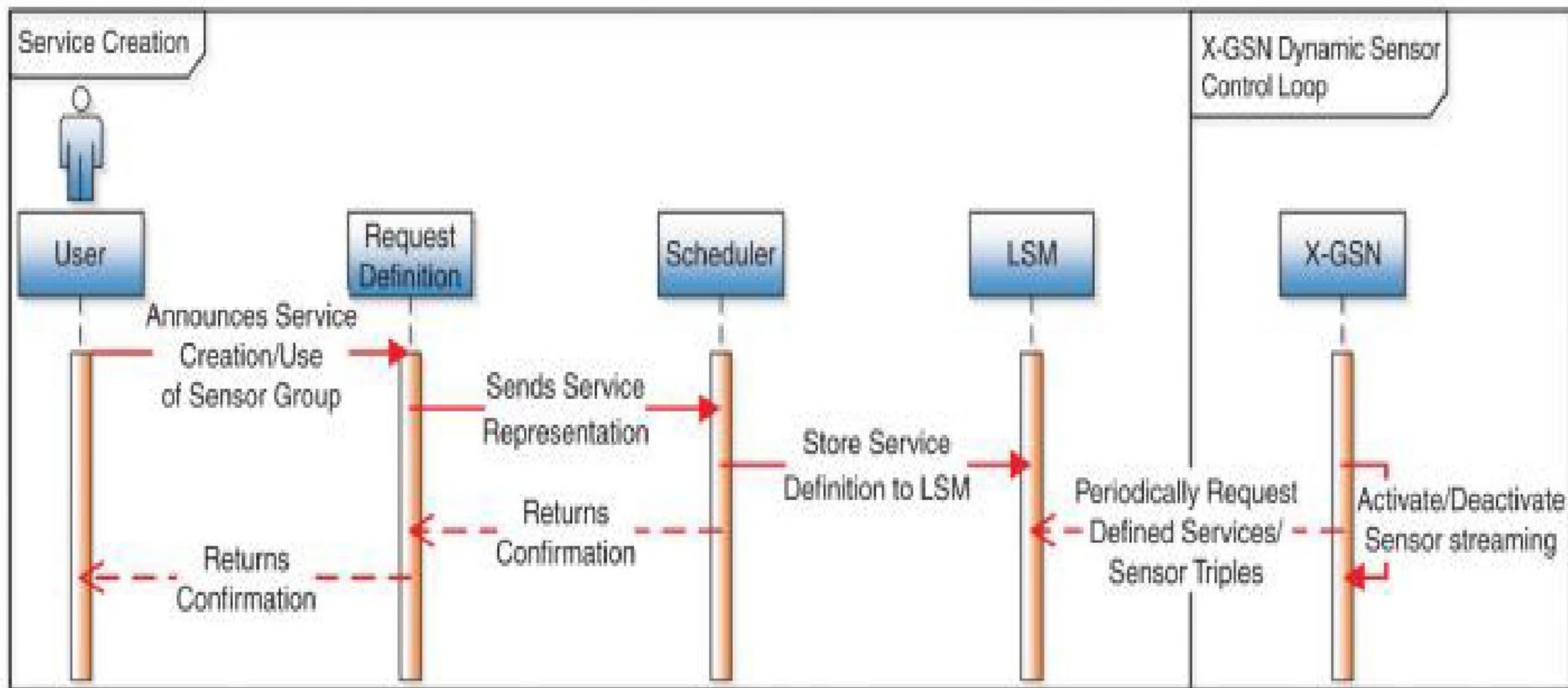
Caching Techniques

- Caching stores sensor data (data streams) in cache memory.
- It provides fast and easy access to the data.
- Caching helps to:
 - Reduce network traffic
 - Improve data availability for users
 - Reduce the cost of cloud access operations (e.g., I/O operations to public clouds)

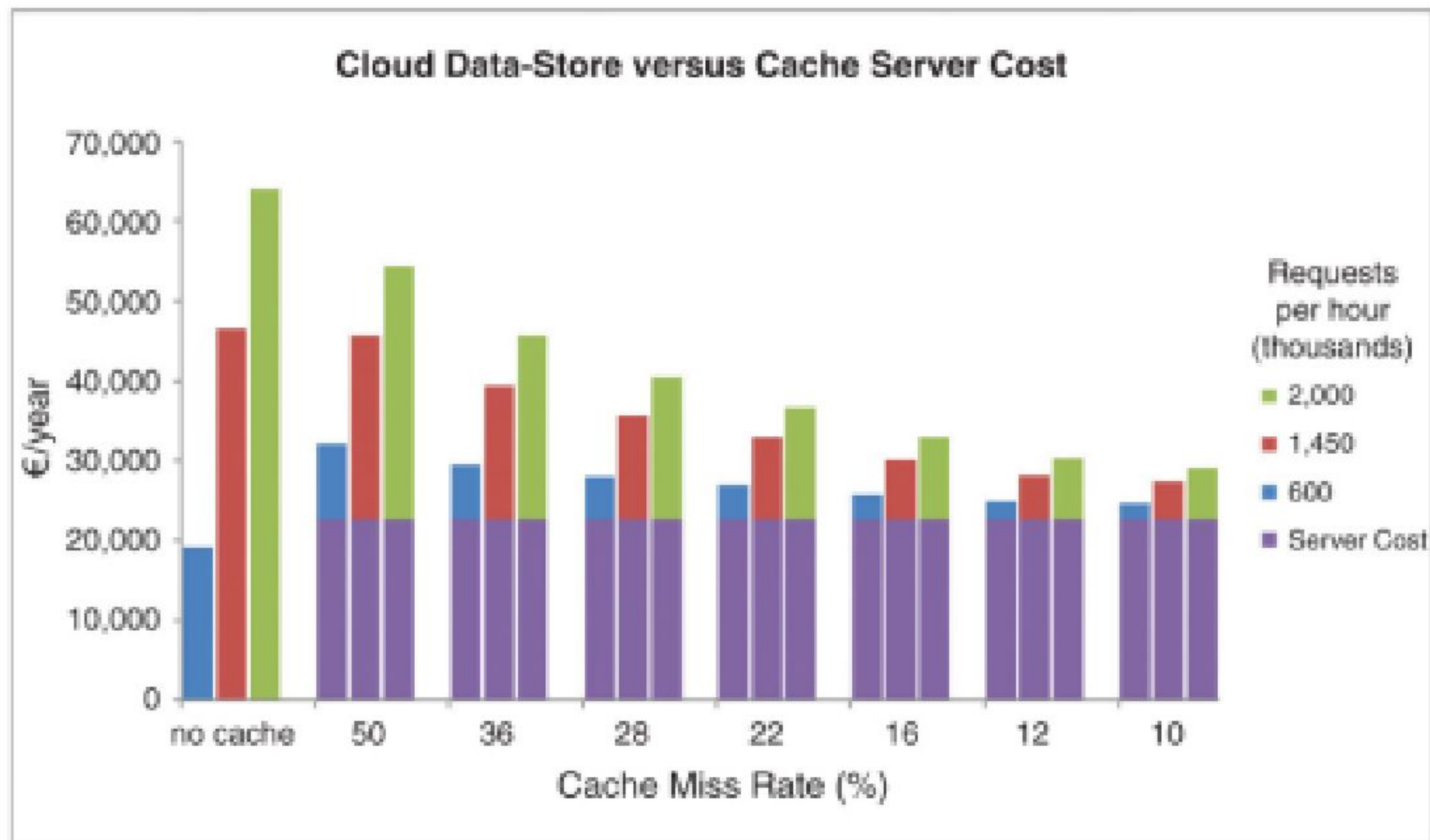
Caching Solution in OpenIoT

- A small proxy layer is used to route all SPARQL queries.
- When a query is received:
 - The proxy layer checks whether the result is already cached.

- If the result is available in the cache:
 - It is returned directly to the client.
 - No access to the SPARQL data store is required.
- If the result is not available in the cache:
 - The query is sent to the SPARQL data store.
 - The result is stored in the local cache.
 - Then, the result is returned to the client.
- An efficient caching solution requires:
 - A clear estimate of the average requests per hour to the cloud data store.
 - Sufficient cache storage capacity.
- Caching provides monetary benefits. It reduces access to remote data stores such as:
 - Amazon S3
 - Google Cloud Data Store
- This helps reduce pay-as-you-go costs based on the cloud provider's pricing scheme.



"Service Creation" Service Flowchart



Comparison of Costs Associated With the Use of Cache Server to the Cost of the Use of a Public Cloud Data-Store

Figure(illustrates the total costs)incurred for seven different scenarios of the SPARQL queries ,that correspond to different varieties of queries, which result in different miss-rates.

- For a low number of requests per hour, there is no benefit for using cache.
- At a medium-high number of hourly requests, such as the second category at 1450 Krph, the threshold where it becomes more efficient to use a caching solution is just hit.
- In the last category at 2000 Krph, that at a high number of requests it is far more efficient to use cache.

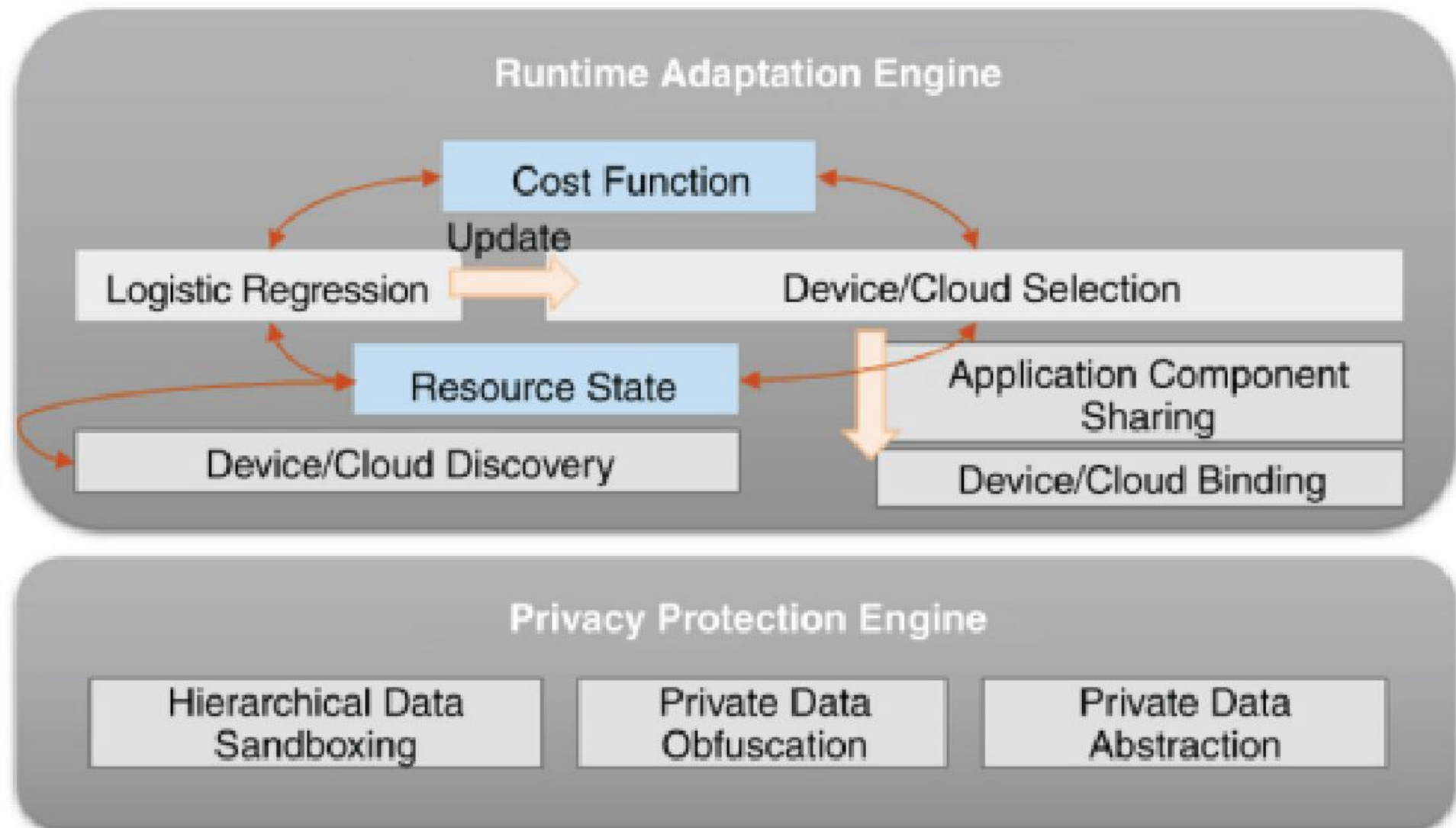
Device/Cloud Collaboration Framework

- Cloud computing provides on-demand computing and storage resources.
- It helps users who cannot afford computing equipment and management costs.
- A Device/Cloud Collaboration Framework is needed to improve:
 - Scalability
 - Privacy protection
- The framework uses the compute and storage resources of smart mobile devices.
- It provides security solutions that protect privacy without reducing application quality.
- It enables collaboration between smart mobile devices and the cloud for scalable and secure computing.

Powerful Smart Mobile Devices

- Modern smartphones have enough computing power to perform various computing tasks.
- However, their usage is limited by:
 - Battery life
 - Network connectivity
- Therefore, cloud resources can be used along with smart mobile devices.

Device/Cloud Collaboration Framework



High-Level Layout of the Device/Cloud Collaboration Framework

Runtime Adaptation Engine

- For a computation task, the framework decides where the task should be executed.
- Example:
 - Processing a voice query using a lightweight voice-query engine on a smartphone.
- The mobile engine can process the query without sending voice data to the cloud.
- However, it consumes the device's battery.
- If the mobile engine returns a poor result:
 - The user may need to send the query again to the cloud-based query processing engine.
 - This can reduce the overall Quality of Experience (QoE).

- A decision mechanism is needed to automatically select the best agent to execute a task.
- The Runtime Adaptation Engine (RAE) is the core of the framework.
- The RAE:
 - Maintains a list of available devices and cloud using Device/Cloud Discovery.
 - Monitors the available resources of devices and the cloud.
- The RAE uses a Logistic Regression algorithm to:
 - Learn the most cost-effective policy.
 - Distribute tasks between devices and the cloud based on the resource state.

- Definition of the cost function is the weighted sum of the resource state, such as:
 - Battery life
 - Network
 - CPU usage
- The Logistic Regression policy is applied by the Device/Cloud Selection module.
- The Device/Cloud Selection module chooses the most economical computing resource for a given task.
- The RAE acts as an autonomous agent.
- It can be deployed on both devices and the cloud.
- RAEs communicate with each other to share resource state and distribute the workload efficiently.

- The cloud-side RAE calculates its cost function using:
 - Residual CPU cycles
 - Available storage space
- If the available capacity falls below a threshold:
 - The cloud rejects the resource-sharing request from devices.
 - This is because executing the task would be too costly.
- The cloud-side RAE advises the device-side RAE to:
 - Execute the task on the device, or
 - Wait until cloud resources become available.
- The RAEs on both devices and the cloud make autonomous decisions.
- They work without any supporting brokerage system.

- The device-side RAE periodically monitors the state of cloud resources.
- The cloud-side RAE does not monitor the resource state of millions of paired devices.
- The cloud-side RAE makes a simple decision:
 - Accept the task-execution request, or
 - Reject the task-execution request.
- By default, the framework does not offload cloud-initiated tasks to devices.

Device/Cloud Discovery Component

- Supports task distribution among neighbouring devices.
- Implements the discovery of devices.

Application Component Sharing

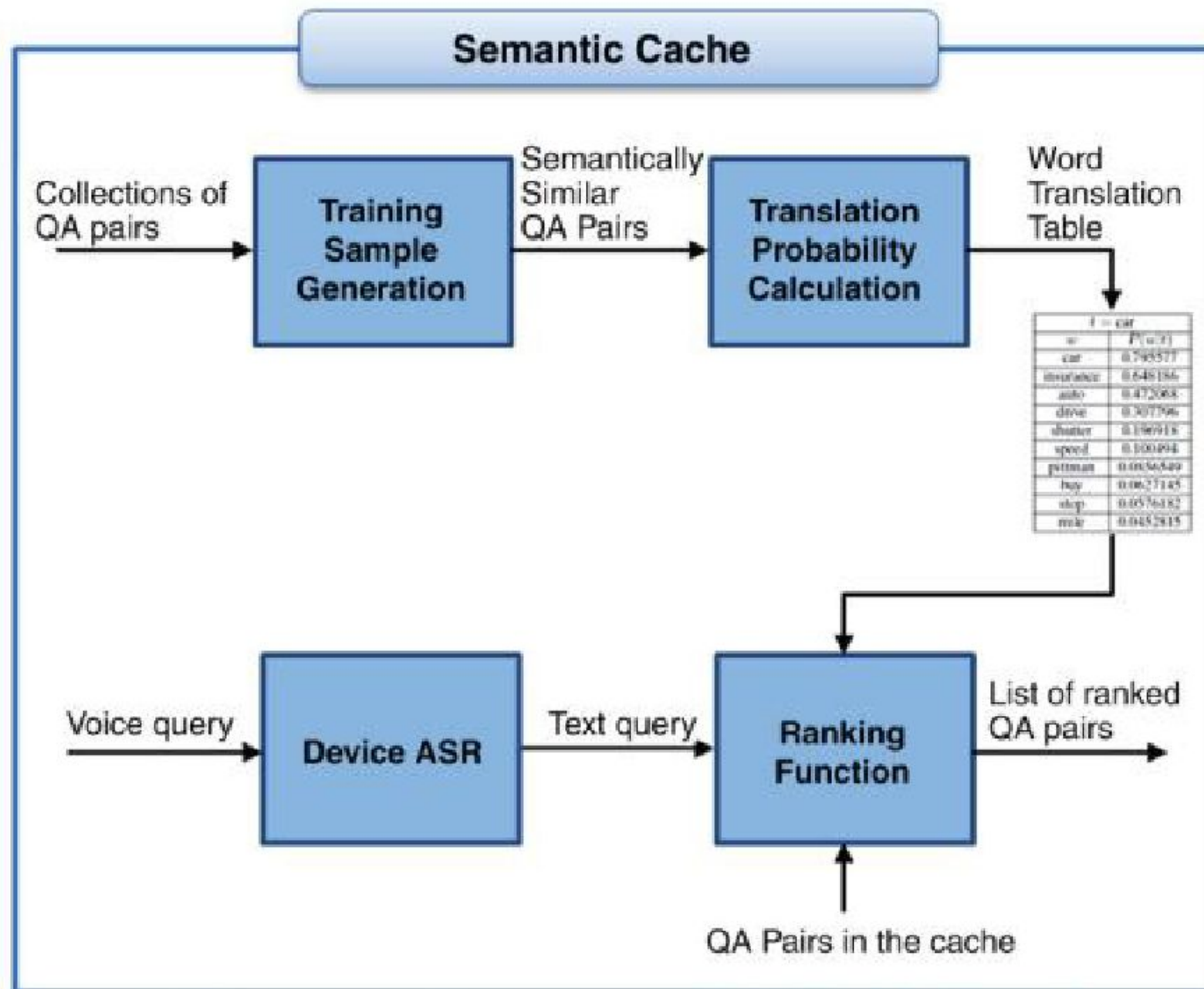
- **Example:** Device A wants to collaborate with Device B.
- If Device B does not have the required application components to process the task:
 - Device A transfers the required application components to Device B.
- This is done through Application Component Sharing.

Applications of Device/Cloud Collaboration

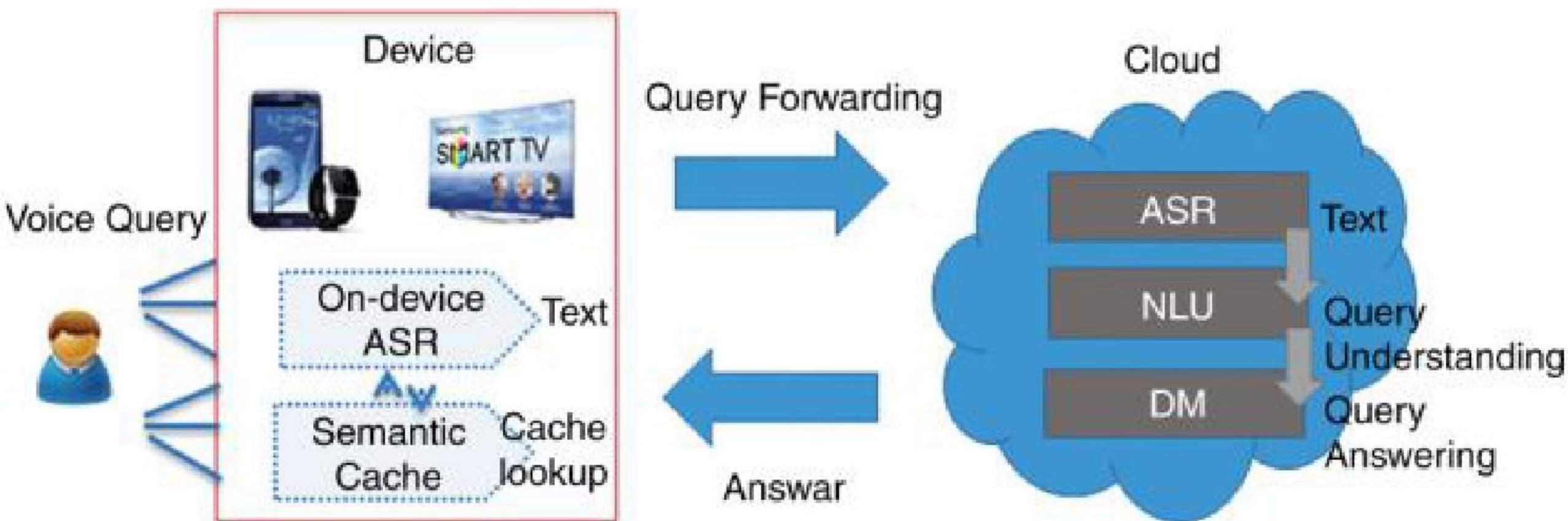
- The Device/Cloud Collaboration Framework is used in real-world intelligent applications developed at Samsung Electronics.
- These applications provide:
 - Context-aware proactive suggestions
 - Semantic QA caching
 - Automatic image recognition
 - Automatic speech recognition

Semantic QA Caching

- Semantic QA Cache is a mobile application.
- It retrieves answers from the cache using semantically similar past queries.
- It is useful when:
 - Internet connectivity is unavailable.
 - The user does not want to send private queries to the cloud.
- It returns:
 - A list of similar queries.
 - Their associated answers.
- It continuously updates the ranking function using the word-translation table.
- The ranking function measures the similarity between a new query and previous queries.



Illustrations of the Technique to Cluster Semantically Similar QA Pairs for Retrieving an Answer for a Newly Given Query Without Asking the QA Engine on the Cloud Side



Semantic QA Cache Implementing the Device/Cloud Collaboration Framework

QUESTIONS

1. Explain all active threats in IOT Security.
 2. Write all the advantages associated with Fog Computing.
 3. Discuss about computation offloading in IoT.
 4. Describe the two dominant IoT architectures.
 5. Explain IoT and describe its major application.
 6. Illustrate the working of Runtime Adaptation Engine within a device-cloud collaboration framework.
 7. Explain about Resource Management in IOT.
 8. Define Caching technique.
 9. Discuss some Applications of IoT in real life scenario.
 10. Compare the features of Cloud and Fog approaches in current data-analytics.
-
1. Explain the taxonomy of resource management in IoT.
 2. Draw and explain the state diagram of the open IoT service life cycle.
 3. Explain about Internet of Things. Illustrate the different Applications of IOT.
 4. With the help of neat diagram explain about the Open IoT Architecture for IoT/Cloud Convergence.
 5. What are the major elements of IoT architecture for IoT/cloud convergence?
 6. With the help of neat diagram explain OpenIOT Architecture for IOT/Cloud Convergence.
 7. With the help of neat diagram explain state diagram of OpenIOT services Lifecycle.